



AMAL JYOTHI
COLLEGE OF ENGINEERING
A U T O N O M O U S

FIRE HYDRANT NETWORK OPTIMIZATION

20INMCA302,20INMCA304,20INMCA306,20INMCA308

Combined Assignment

Ms. Meera Rose Mathew

Ms. Lisha Varghese

Dr. Shelly Shiju George

Ms. Sona Maria Sebastian

Assistant Professor's Department of Computer Application

**DEPARTMENT OF
COMPUTER APPLICATIONS**



AMAL BABU
AJC22MCA-I014
INMCA2022-27 S6

ABSTRACT

The **Fire Hydrant Network Optimization System** utilizes **Prim's Algorithm** to enhance emergency response efficiency by determining the **optimal placement and connectivity of fire hydrants**. The system processes hydrant location data from a **CSV file**, constructs an **adjacency matrix**, and applies **graph-based optimization** to generate a **Minimum Spanning Tree (MST)** that minimizes the total distance between hydrants. By reducing redundancy and ensuring an optimized water supply network, this solution aids in improving **firefighting preparedness** while lowering infrastructure costs. The system provides a **user-friendly VB.NET interface**, allowing users to **import data, visualize results, and analyze cost-effective hydrant connections**, ensuring faster and more efficient emergency response.



ABOUT THE DATASET

The dataset for the **Fire Hydrant Network Optimization** project contains crucial information about fire hydrants and their spatial distribution, water pressure levels, and connectivity costs. This dataset is used to optimize the placement and network of fire hydrants for effective emergency response.

It includes fields like **Hydrant ID**, **Latitude**, **Longitude**, **Water Pressure (psi)**, **Distance to Next Hydrant (meters)**, and **Incident Risk Level**.

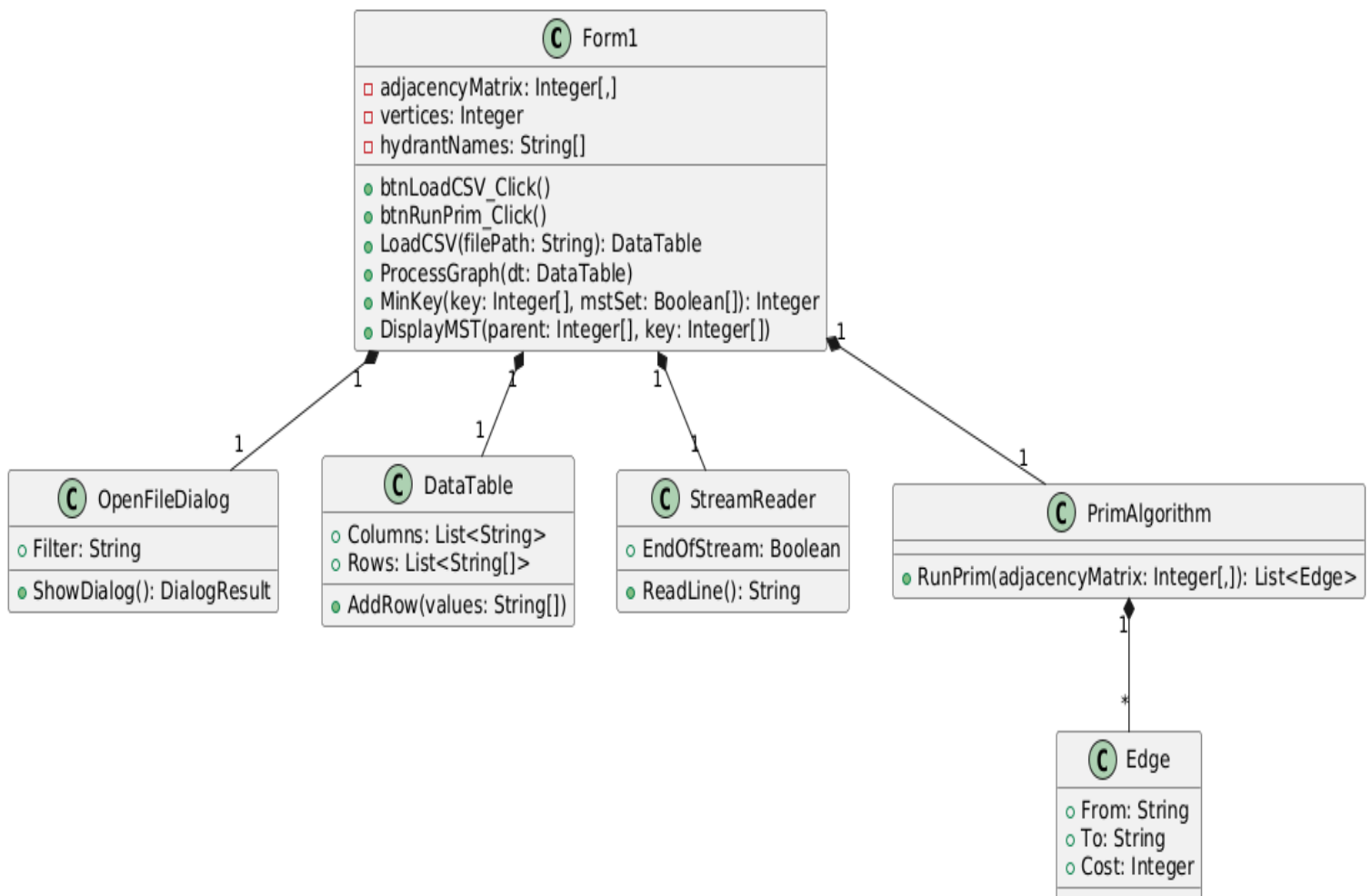
This data helps in designing an optimized fire hydrant network using **Prim's Algorithm** by identifying the shortest and most cost-effective connections between hydrants. The goal is to minimize infrastructure costs while ensuring optimal coverage and accessibility for firefighting operations.

The dataset provides essential insights for analyzing geographical, hydraulic, and risk factors, aiding in the development of a well-balanced and efficient fire hydrant system.

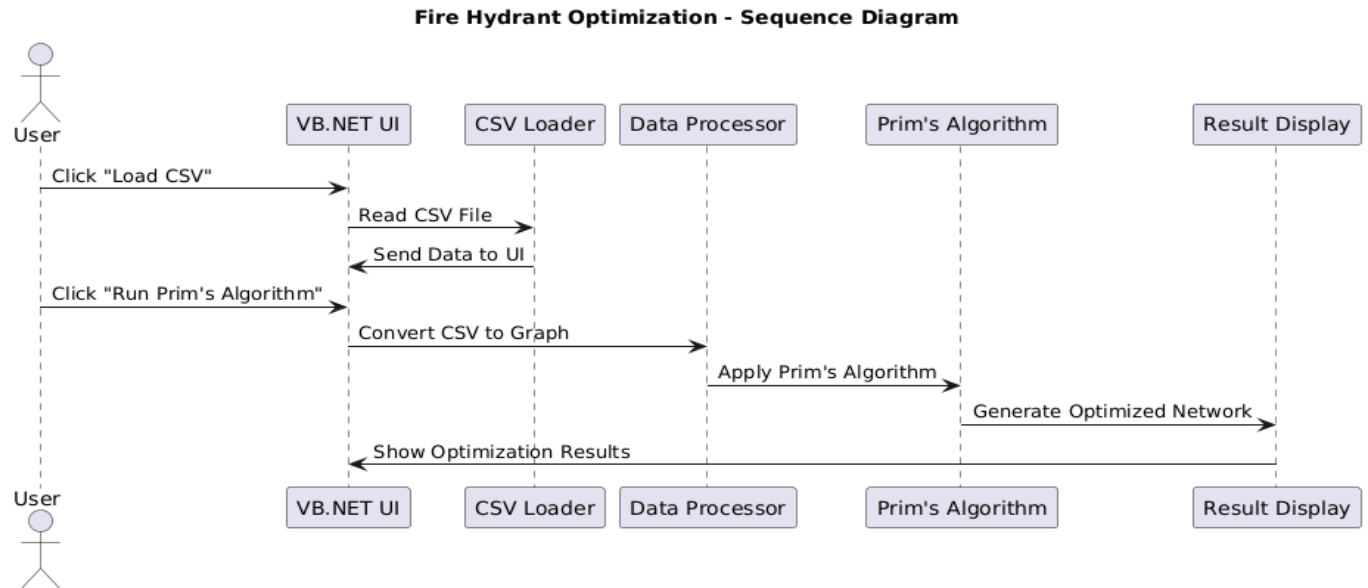
Hydrant ID	Latitude	Longitude	Water Pressure (psi)	Distance to Next Hydrant (meters)	Incident Risk Level
H1	9.5915	76.5221	50	150	High
H2	9.5932	76.5245	45	200	Medium
H3	9.5951	76.5268	55	180	Medium
H4	9.5973	76.5292	60	220	High
H5	9.599	76.531	52	140	Low
H6	9.6005	76.5334	47	160	Medium
H7	9.6031	76.5358	58	190	High
H8	9.6057	76.5381	53	210	Medium
H9	9.6083	76.5405	50	170	Low

UML DIAGRAM

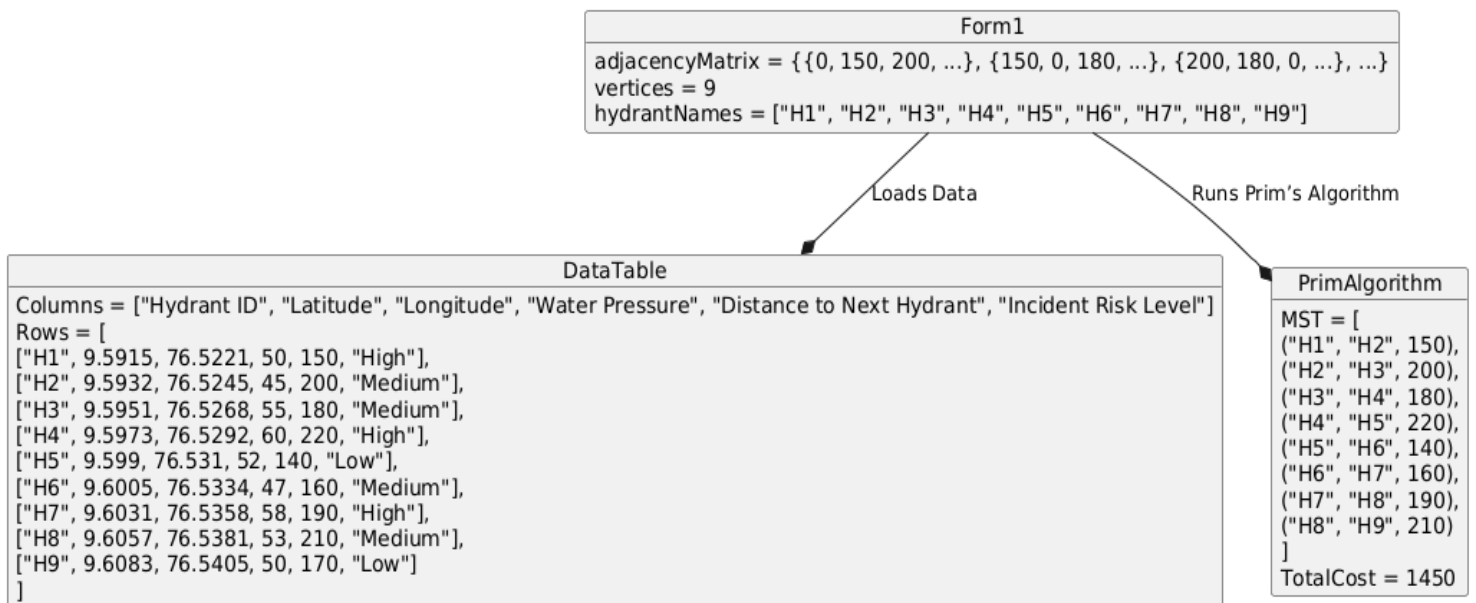
CLASS DIAGRAM:



SEQUENCE DIAGRAM:

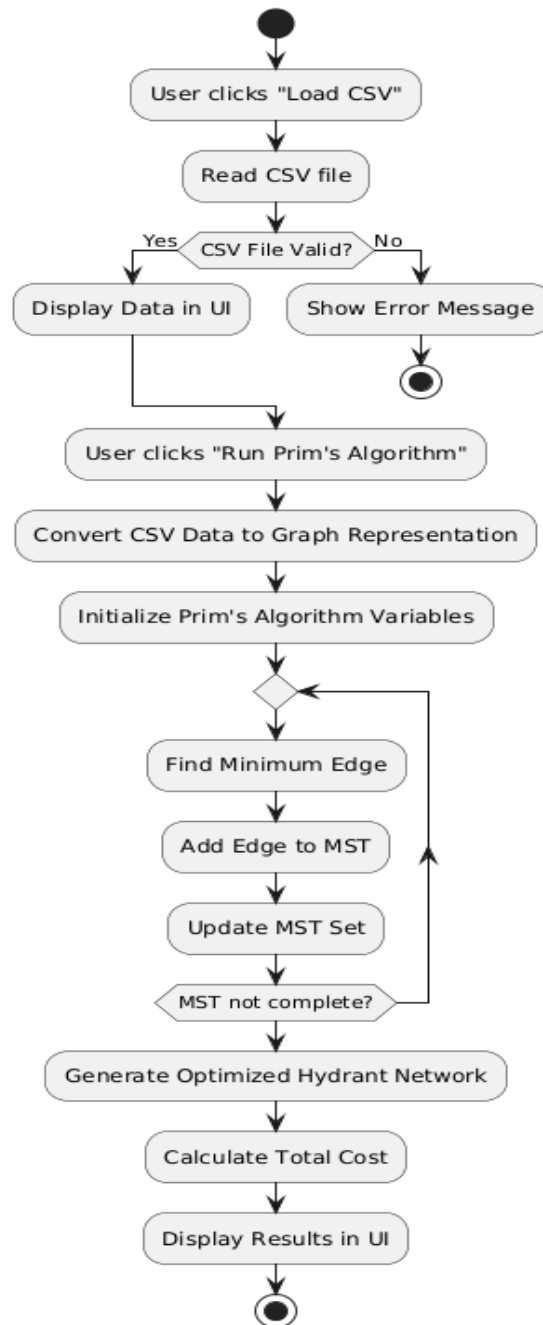


OBJECT DIAGRAM:



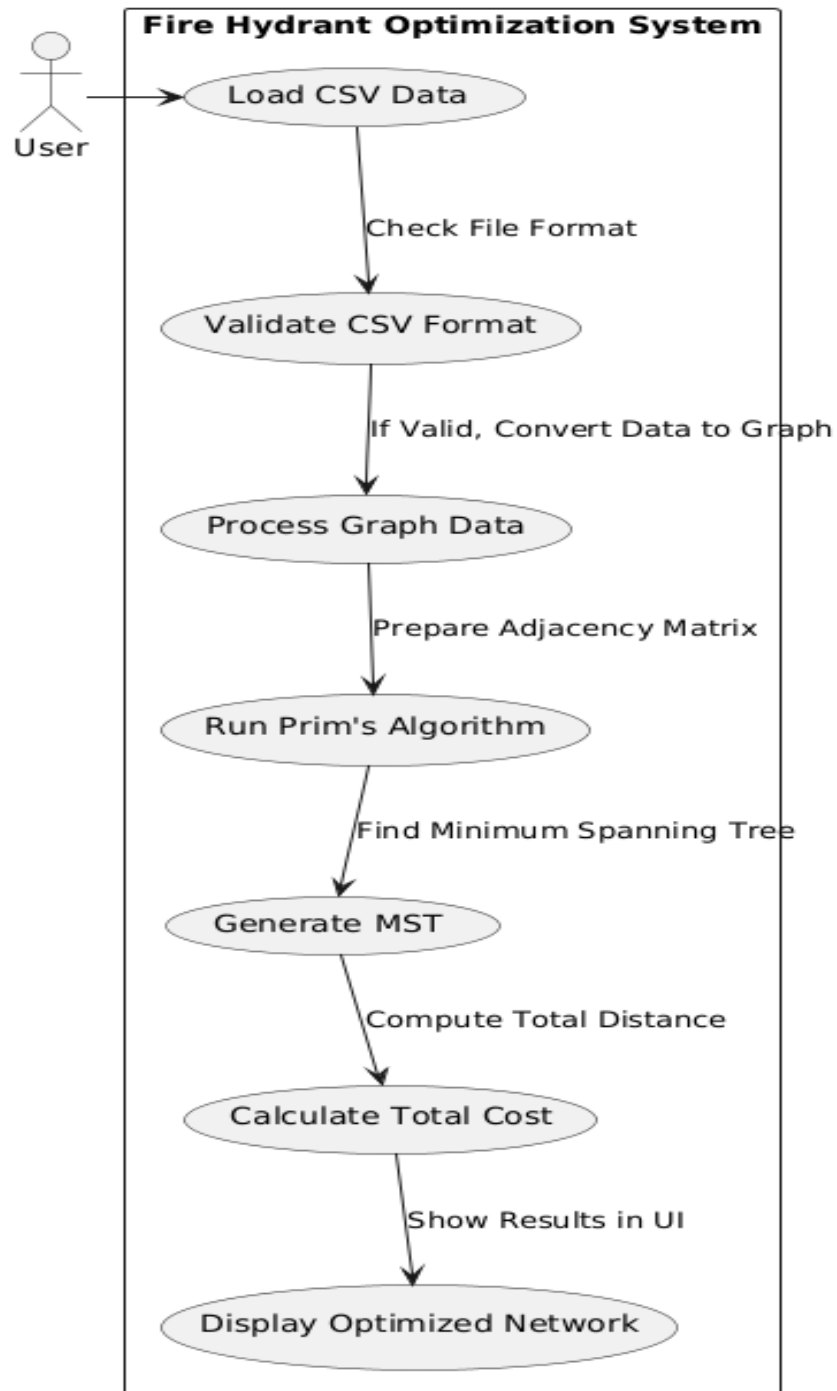
ACTIVITY DIAGRAM:

Fire Hydrant Optimization - Activity Diagram



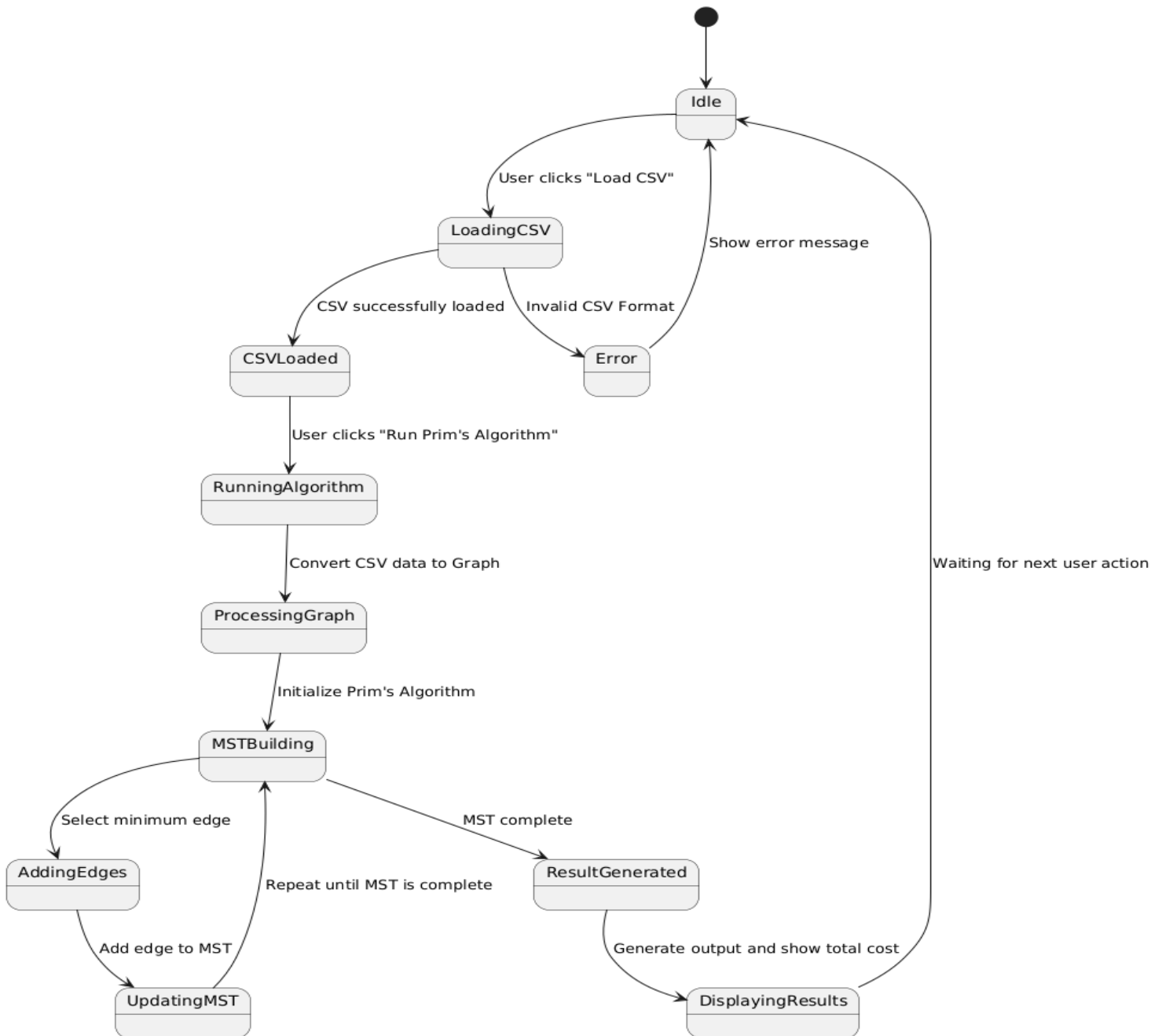
USE CASE DAIGRAM:

Fire Hydrant Optimization - Use Case Diagram



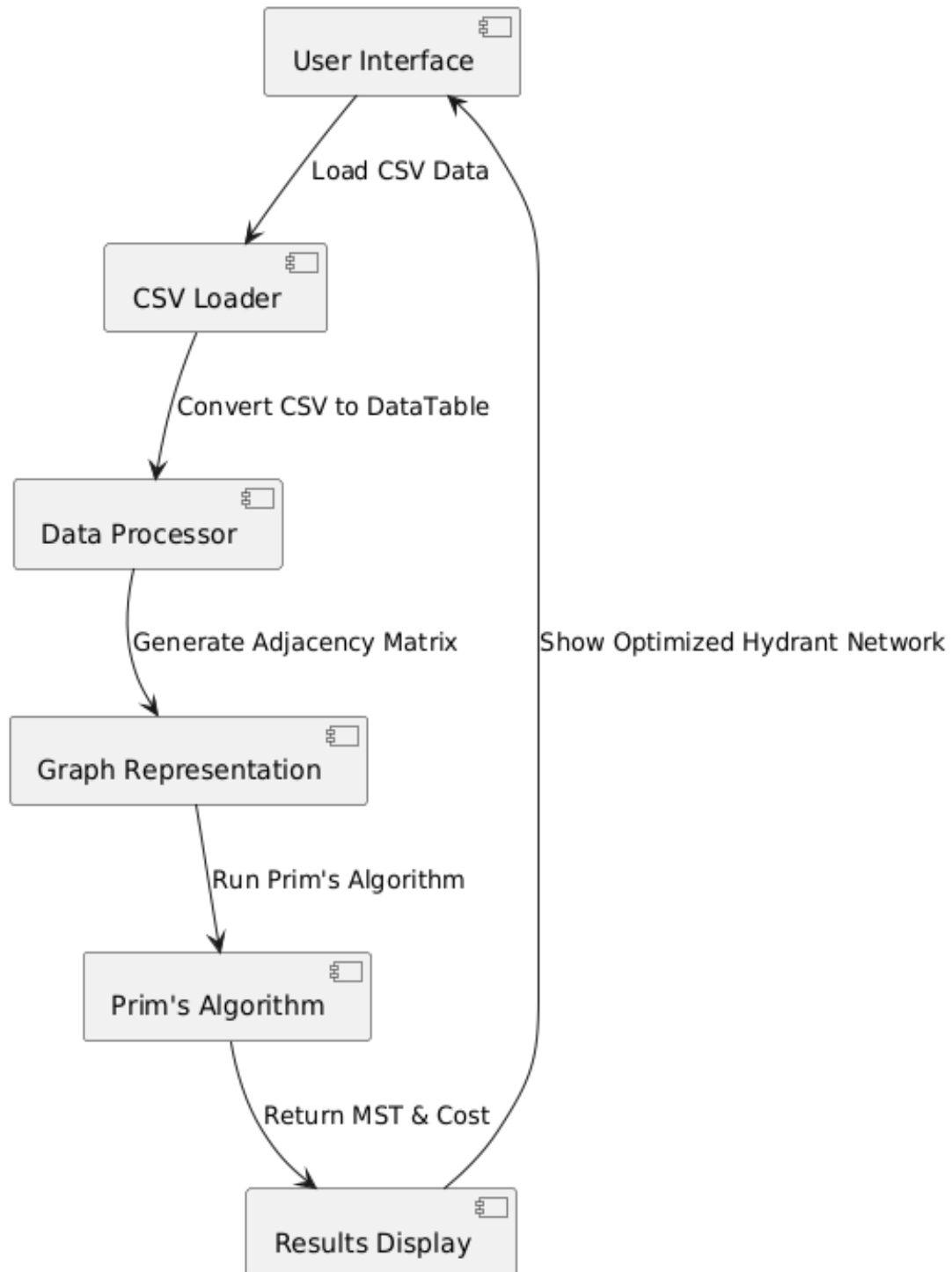
STATE CHART DIAGRAM:

Fire Hydrant Optimization - State Chart



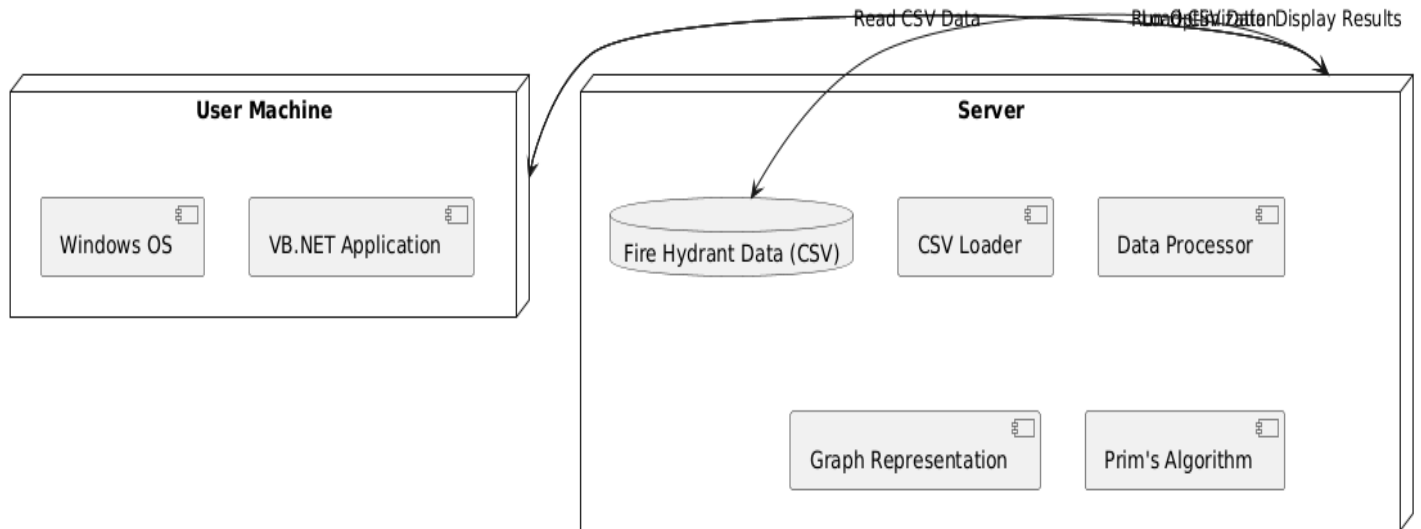
COMPONENT DIAGRAM:

Fire Hydrant Optimization - Component Diagram



DEPLOYMENT DIAGRAM:

Fire Hydrant Optimization - Deployment Diagram



CODE

```
Imports System.IO
Imports System.Data

Public Class Form1

    Private adjacencyMatrix(,) As Integer
    Private vertices As Integer
    Private hydrantNames() As String ' Store hydrant names

    ' Load CSV Data
    Private Sub btnLoadCSV_Click(sender As Object, e As EventArgs) Handles btnLoadCSV.Click
        Dim openFileDialog As New OpenFileDialog()
        openFileDialog.Filter = "CSV Files (*.csv)|*.csv|All Files (*.*)|*.*"

        If openFileDialog.ShowDialog() = DialogResult.OK Then
            Dim filePath As String = openFileDialog.FileName
            Dim dt As DataTable = LoadCSV(filePath)
            DataGridView1.DataSource = dt
            ProcessGraph(dt)
        End If
    End Sub

    ' Load CSV into DataTable
    Private Function LoadCSV(filePath As String) As DataTable
        Dim dt As New DataTable()

        Try
            Using reader As New StreamReader(filePath)
                If Not reader.EndOfStream Then
                    Dim headers = reader.ReadLine().Split(",")
```

```

    For Each header In headers
        dt.Columns.Add(header.Trim())
    Next
End If

```

```

While Not reader.EndOfStream
    Dim values = reader.ReadLine().Split(", "c)
    dt.Rows.Add(values)
End While
End Using
Catch ex As Exception
    MessageBox.Show("Error loading CSV: " & ex.Message)
End Try

```

```

Return dt
End Function

```

' Convert CSV Data to Adjacency Matrix

Private Sub ProcessGraph(dt As DataTable)

```

    vertices = dt.Rows.Count
    ReDim adjacencyMatrix(vertices - 1, vertices - 1)
    ReDim hydrantNames(vertices - 1)

```

' Store Hydrant Names

```

For i As Integer = 0 To vertices - 1
    hydrantNames(i) = dt.Rows(i)("Hydrant ID").ToString()
Next

```

' Initialize adjacency matrix with distances

```

For i As Integer = 0 To vertices - 1

```

```

For j As Integer = 0 To vertices - 1
    If i <> j Then
        Dim dist As Integer = ComputeDistance(dt, i, j)
        adjacencyMatrix(i, j) = dist
        adjacencyMatrix(j, i) = dist ' Ensure symmetry
    Else
        adjacencyMatrix(i, j) = 0 ' No self-loops
    End If
Next
Next
End Sub

' Compute Distance between Two Hydrants
Private Function ComputeDistance(dt As DataTable, i As Integer, j As Integer) As Integer
    Dim lat1 As Double = Convert.ToDouble(dt.Rows(i)("Latitude"))
    Dim lon1 As Double = Convert.ToDouble(dt.Rows(i)("Longitude"))
    Dim lat2 As Double = Convert.ToDouble(dt.Rows(j)("Latitude"))
    Dim lon2 As Double = Convert.ToDouble(dt.Rows(j)("Longitude"))

    Return Math.Sqrt((lat2 - lat1) ^ 2 + (lon2 - lon1) ^ 2) * 111000 ' Convert to meters
End Function

' Run Prim's Algorithm
Private Sub btnRunPrim_Click(sender As Object, e As EventArgs) Handles btnRunPrim.Click
    If vertices = 0 Then
        MessageBox.Show("Load CSV data first!")
        Return
    End If

    Dim parent(vertices - 1) As Integer

```



```

Dim key(vertices - 1) As Integer
Dim mstSet(vertices - 1) As Boolean

' Initialize keys
For i As Integer = 0 To vertices - 1
    key(i) = Integer.MaxValue
    mstSet(i) = False
Next

key(0) = 0
parent(0) = -1

' Build MST
For count As Integer = 0 To vertices - 2
    Dim u As Integer = MinKey(key, mstSet)
    mstSet(u) = True

    For v As Integer = 0 To vertices - 1
        If adjacencyMatrix(u, v) > 0 AndAlso mstSet(v) = False AndAlso adjacencyMatrix(u, v) <
key(v) Then
            parent(v) = u
            key(v) = adjacencyMatrix(u, v)
        End If
    Next
Next

DisplayMST(parent, key)

End Sub

' Find Minimum Key Vertex
Private Function MinKey(ByVal key() As Integer, ByVal mstSet() As Boolean) As Integer

```

```

Dim min As Integer = Integer.MaxValue
Dim minIndex As Integer = -1

For v As Integer = 0 To vertices - 1
    If mstSet(v) = False AndAlso key(v) < min Then
        min = key(v)
        minIndex = v
    End If
Next

Return minIndex

End Function

' Display Minimum Spanning Tree
Private Sub DisplayMST(ByVal parent() As Integer, ByVal key() As Integer)
    lstMSTResult.Items.Clear()
    lstMSTResult.Items.Add("Optimized Hydrant Network:")
    lstMSTResult.Items.Add("-----")

    Dim totalCost As Integer = 0
    For i As Integer = 1 To vertices - 1
        lstMSTResult.Items.Add(hydrantNames(parent(i)) & " - " & hydrantNames(i) & " Cost: " &
key(i))
        totalCost += key(i)
    Next

    lstMSTResult.Items.Add("-----")
    lstMSTResult.Items.Add("Total Cost: " & totalCost)

End Sub

End Class

```

OUTPUT

Result: Question have been successfully executed and output verified

	Hydrant ID	Latitude	Longitude	Water Pressure (psi)	Distance to Next Hydrant (meters)	Incident Risk Level
▶	H1	9.5915	76.5221	50	150	High
	H2	9.5932	76.5245	45	200	Medium
	H3	9.5951	76.5268	55	180	Medium
	H4	9.5973	76.5292	60	220	High
	H5	9.599	76.531	52	140	Low
	H6	9.6005	76.5334	47	160	Medium
	H7	9.6031	76.5358	58	190	High
	H8	9.6057	76.5381	53	210	Medium
	H9	9.6083	76.5405	50	170	Low
*						

Load CSV

Run Prim's Algorithm

Fire Hydrant Optimization

Optimized Hydrant Network:

H1 - H2 Cost: 326
H2 - H3 Cost: 331
H3 - H4 Cost: 361
H4 - H5 Cost: 275
H5 - H6 Cost: 314
H6 - H7 Cost: 393
H7 - H8 Cost: 385
H8 - H9 Cost: 393

Total Cost: 2778